

1. Project Overview

Project Title:

UnicomTic Management System

2.Key Features Implemented:

Role-Based Login System:

The application uses hardcoded default credentials (Username: admin, Password: admin123, Role: admin) for first-time access.

After the initial login, the admin can create users for various roles. From that point forward, system access should be through the credentials created via the User Management Form.

Role-Based Dashboards:

Admin: Full access to all modules – manage users, subjects, courses, exams, marks, timetables, rooms, and roles (Admin, Student, Lecturer, Staff).

Lecturer: Can manage student marks and view their timetable.

Student: Can view only their own marks,based on their userId and timetable.

Staff: Can manage courses, timetables, and student data.

User Management Module:

Admin can add users with specific roles and associated details such as username, password, NIC, gender, phone, email, and course.

Based on the selected role, the user data is automatically categorized and stored under Student, Lecturer, Staff, or Admin.

Form-Based Module Handling:

Each module (Subjects, Exams, Courses, Marks, Timetable, Rooms) is developed with its own form for Add, Edit, and Delete functionality.

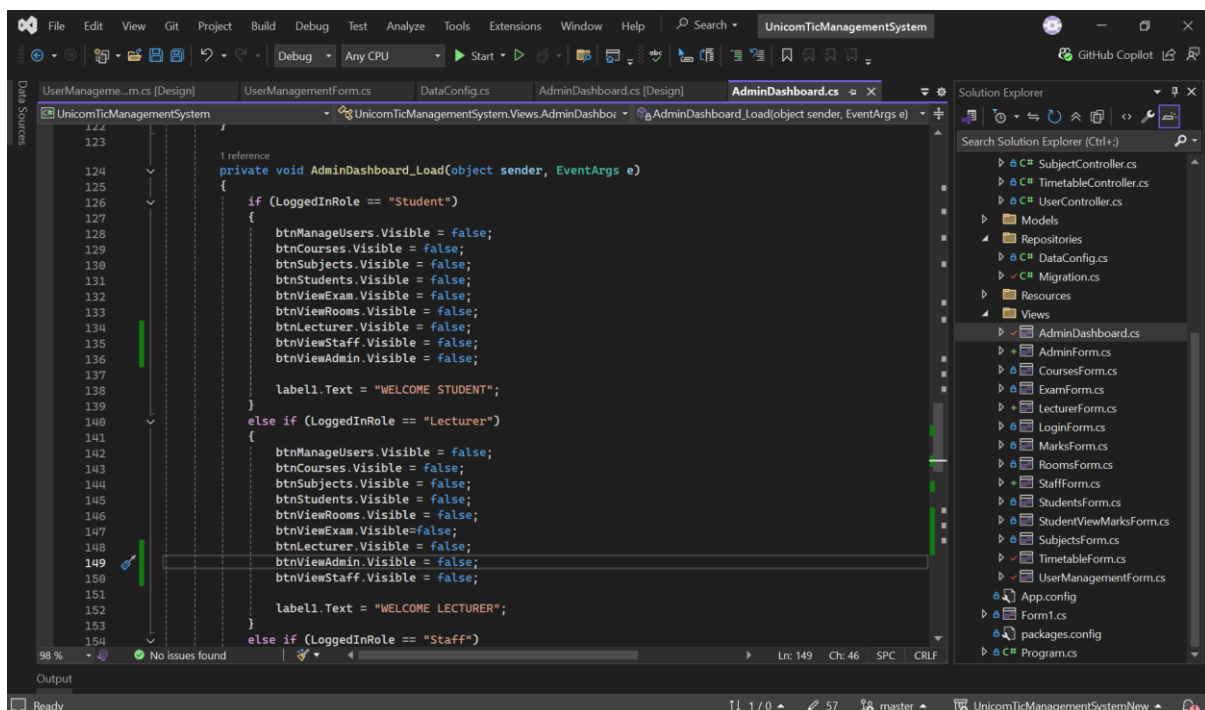
Dynamic UI Rendering:

The dashboard interface dynamically adapts according to the logged-in user's role by controlling button visibility (Visible property in UI), providing a clean and secure user experience.

Authentication and Access Control:

Secure role-based access ensures that users only see and interact with modules relevant to their roles.

Only the Admin can manage users and system-wide features, while other roles are restricted to their specific permissions.



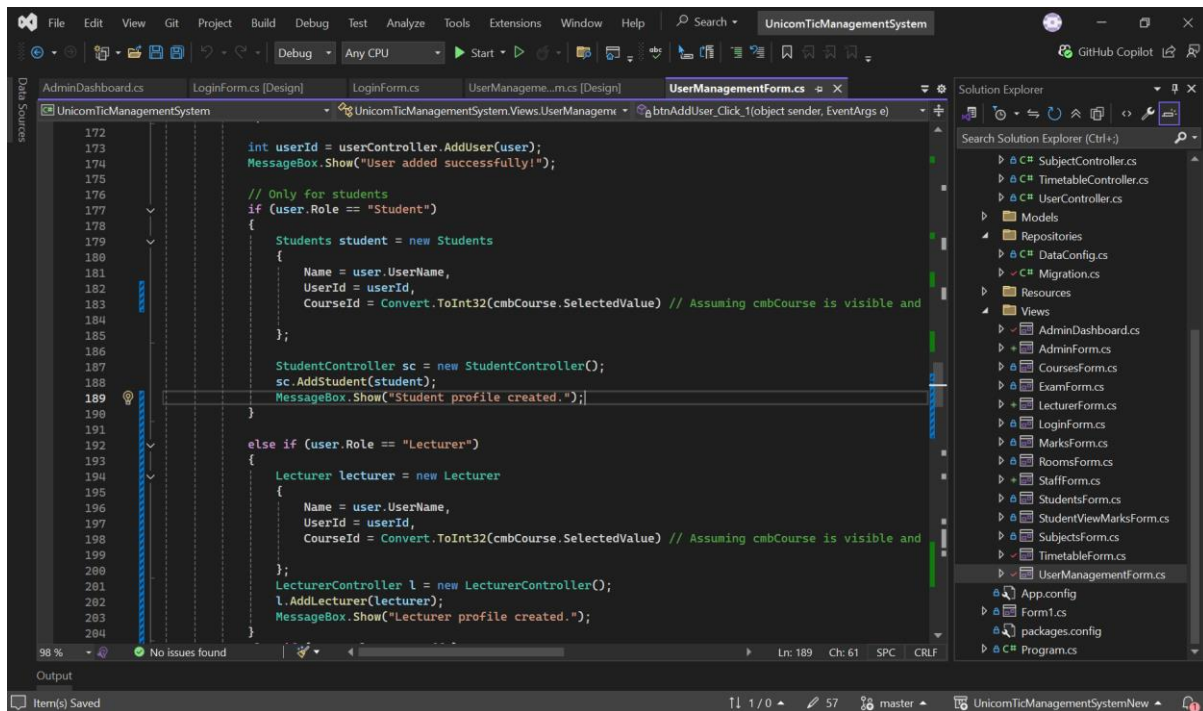
3. Technologies Used:

C# Windows Forms (WinForms) – for user interface design and event handling.

SQLite – for local database storage and management.

Object-Oriented Programming (OOP) – using classes for separation of models, controllers, and UI logic.

ADO.NET with SQLite – for SQL queries and database operations.



Challenges Faced and Solutions:

Challenge: Ensuring dynamic role-based UI controls.

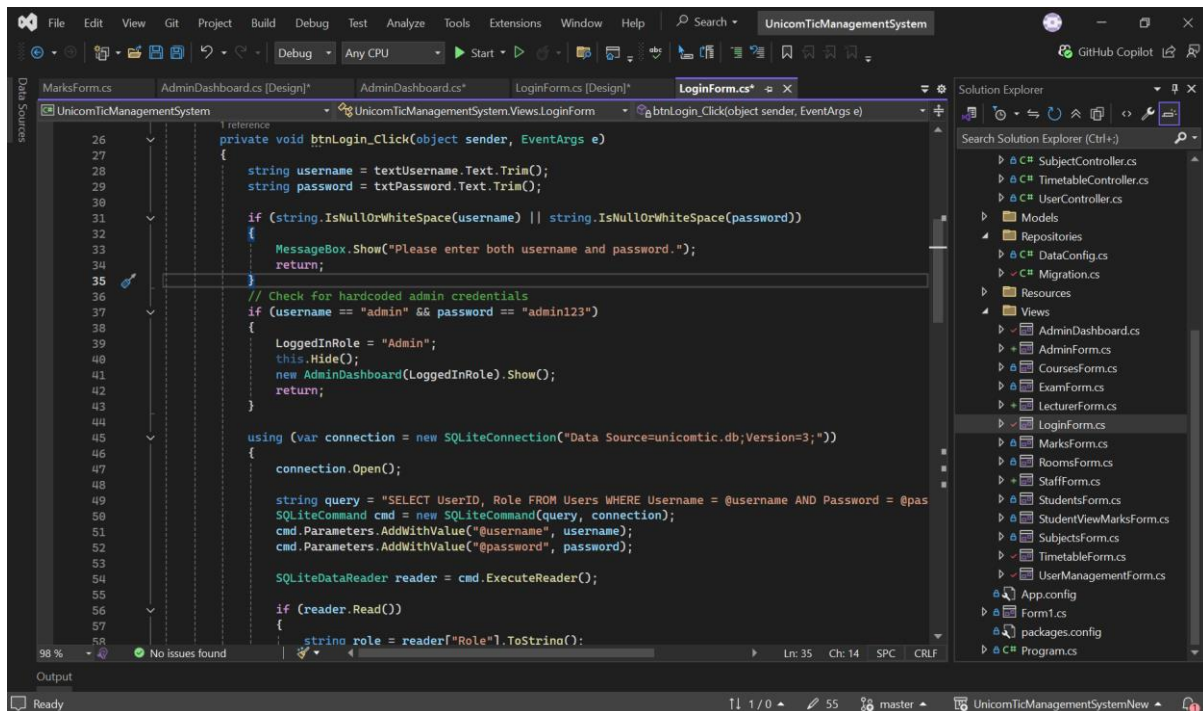
Solution: Used conditional role checks to show/hide specific buttons on dashboard based on the logged-in role.

Challenge: Managing users with multiple roles in a centralized form.

Solution: Implemented a smart ManageUser form that auto-inserts data into different role tables based on selected role using a switch logic.

Challenge: Data binding with multiple forms and table structures.

Solution: Created controller classes (e.g., ExamController, SubjectController) to handle logic and reuse across forms.



Login System:

Admin login is hardcoded for First time. Other users are created and managed through the Admin dashboard.

Password and role-based redirection upon login.

1.Admin Dashboard Includes:

Manage Users (Create/Update/Delete for all roles)

Manage Marks, Exams, Subjects, Courses, Timetable, and Rooms

2.Student View Includes:

View own Marks

View Timetable

3.Lecturer View Includes:

Manage Marks

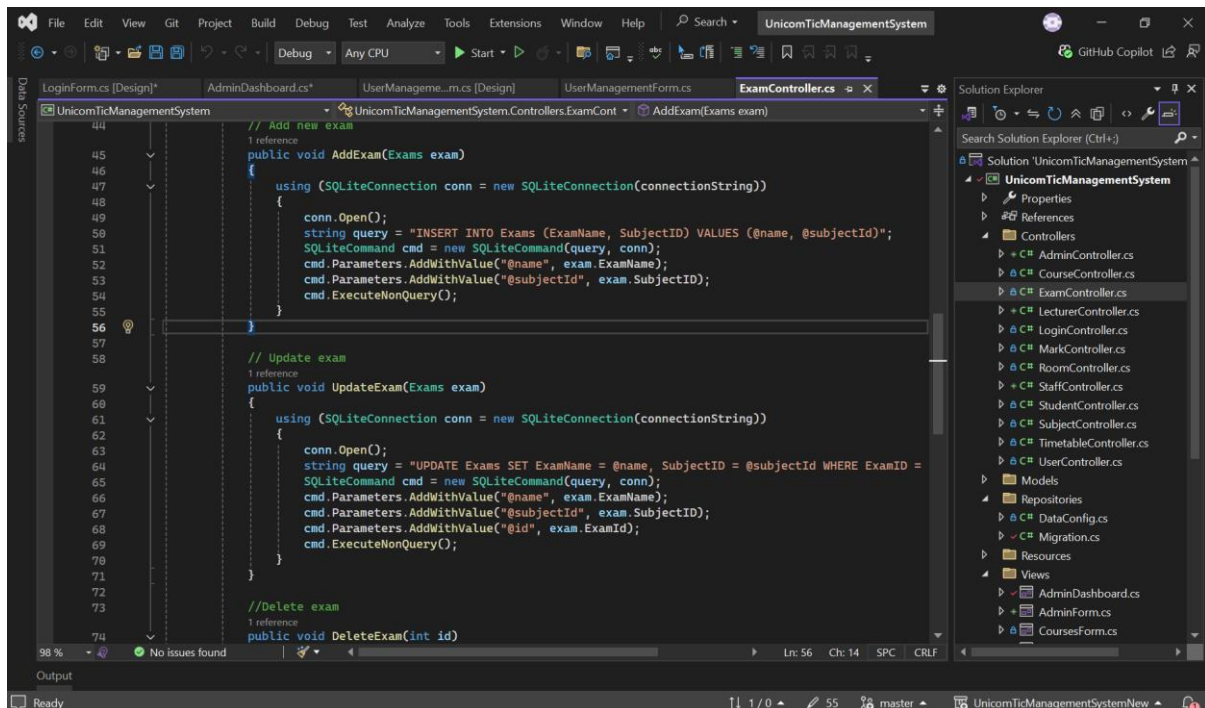
View Timetable

4.Staff View Includes:

Manage Students

Manage Courses

Manage Timetable



OOP Design

Class Separation:

Each data type (Exams, Subjects, Users) has a model class.

Logic is handled in controller classes (e.g., ExamController, UserController).

Encapsulation & Modularity:

Code is organized with encapsulated methods like AddExam(), DeleteUser(), UpdateCourse(), LoadSubjects() to ensure reusability and maintainability.

Dynamic Form Handling:

Each role-based form is separate, using role checks and modular logic.

4. Database Management

Database:

SQLite is used as the backend database.

Multiple tables: Users, Exams, Subjects, Courses, Timetable, Marks, Rooms.

CRUD Operations:

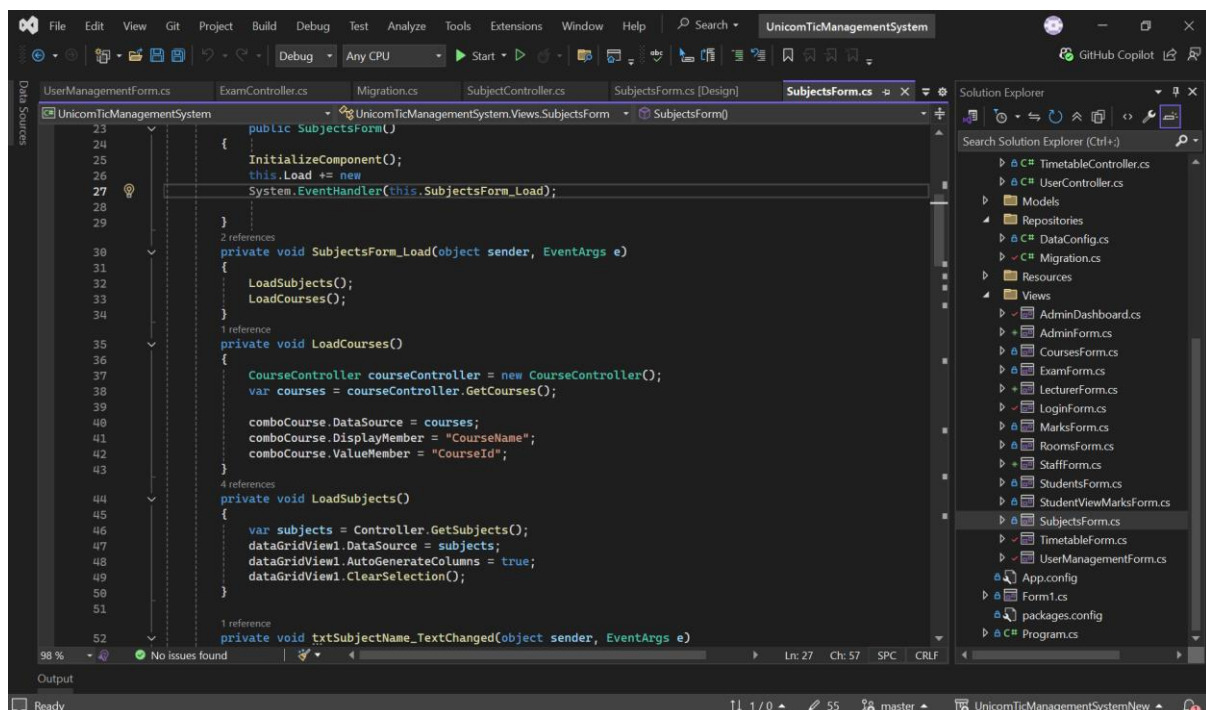
Add/Update/Delete using parameterized SQL queries.

Used SQLiteCommand, SQLiteConnection, SQLiteDataReader for secure operations.

Data Security:

Avoided SQL injection by using @parameter placeholders in all queries.

Data Binding:



Loaded subjects/courses in ComboBox dynamically.

Displayed data in DataGridView controls across all forms.

Table Relationship

The following table describes the relationship between key tables

In the Unicom Management System database:

Table	Links To	Relationship Type
Students	Users, Courses	Many-to-One
Lecturer	Users, Courses	Many-to-One
Staff	Users	Many-to-One
Admin	Users	Many-to-One
Subjects	Courses	Many-to-One
Exams	Subjects	Many-to-One
Marks	Students, Exams	Many-to-One
Timetables	Subjects, Rooms	Many-to-One

5. Error Handling

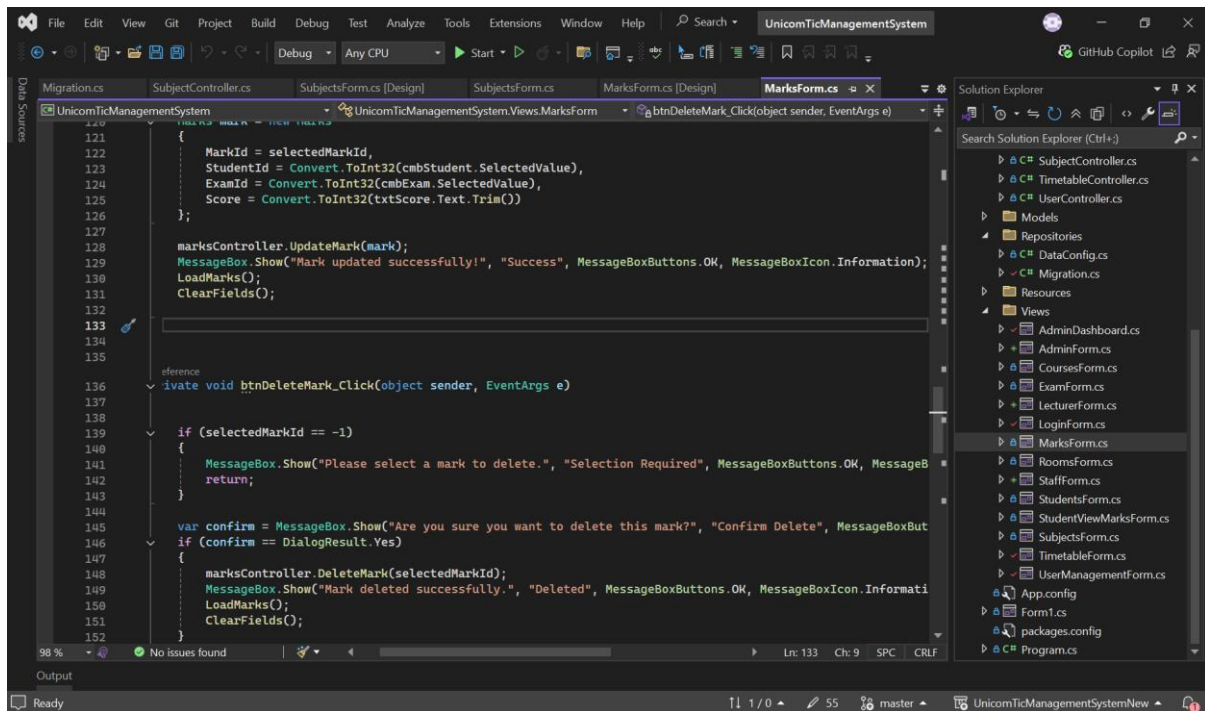
Validation:

Input fields are checked for empty/null values using `string.IsNullOrEmpty()` before processing.

Safe Data Access:

Used `Convert.ToInt32()`, `ToString()` safely when accessing database or `DataGridView` values.

User Prompts:



Confirmations shown before deleting records using `MessageBox.Show()` with Yes/No.

Role-based UI Protection:

Only relevant controls are enabled for each role to prevent unauthorized actions.

6.Future Enhancements

User Profile View:

Each user (Student, Lecturer, Staff, Admin) will have access to view and manage their own profile information such as phone number, email, and personal details.

Personalized Timetable View:

Users will be able to view a customized timetable based on their enrolled subjects or teaching schedule.

Secure Password Change Option:

A feature will be added to allow users to securely update their passwords.

Search and Filter Options:

Introduce search and filtering functionalities for efficient navigation of users, subjects, or marks records.

Attendance Management Module:

Lecturers and staff will be able to mark student attendance. Students can view their own attendance records, and admins can generate attendance reports for analysis.